

Exemplos de Padrões de Projetos (GoF) Criacionais

1. Padrão Singleton

Garante que uma classe possua apenas uma instância global, fornecendo um ponto único de acesso a ela. Essencial para controle de recursos compartilhados e custosos.

Cenário 1.1: Pool de Conexões com Banco de Dados

Contexto: Em aplicações de alta concorrência, abrir uma nova conexão com o banco de dados para cada transação esgota as portas TCP do servidor. Um Pool de Conexões gerencia um número fixo de conexões abertas. Se existissem múltiplas instâncias do Pool, o limite de conexões seria violado, derrubando o banco.

Java

```
public class DatabaseConnectionPool {
    private static volatile DatabaseConnectionPool instancia;

    private DatabaseConnectionPool() {
        System.out.println("Iniciando as 10 conexões físicas com o banco de dados...");
    }

    public static DatabaseConnectionPool getInstancia() {
        if (instancia == null) {
            synchronized (DatabaseConnectionPool.class) {
                if (instancia == null) {
                    instancia = new DatabaseConnectionPool();
                }
            }
        }
        return instancia;
    }

    public void executarQuery(String sql) {
        System.out.println("Alocando conexão do pool para executar: " + sql);
    }
}
```

```
}
```

Cenário 1.2: Gerenciador de Filas de Mensageria (Message Broker)

Contexto: Em uma arquitetura orientada a eventos, diversos módulos publicam mensagens em um barramento (ex: RabbitMQ). A classe responsável por manter o canal aberto com o *broker* deve ser única para evitar conexões zumbis e garantir o enfileiramento sequencial correto.

Java

```
public class MessageBrokerManager {
    // Implementação Eager (Inicialização antecipada)
    private static final MessageBrokerManager INSTANCIA = new MessageBrokerManager();

    private MessageBrokerManager() {
        System.out.println("Estabelecendo canal TCP único com o Message Broker.");
    }

    public static MessageBrokerManager getInstancia() {
        return INSTANCIA;
    }

    public void publicarMensagem(String topico, String payload) {
        System.out.println("Publicando no tópico [" + topico + "]: " + payload);
    }
}
```

Cenário 1.3: Cache de Configurações de Sistema

Contexto: Sistemas corporativos carregam chaves de API e variáveis de ambiente na inicialização. Um serviço Singleton garante que o arquivo `application.properties` seja lido do disco rígido apenas uma vez (mitigando I/O), mantendo os dados em memória para acesso ultrarrápido por qualquer componente.

Java

```
import java.util.HashMap;
import java.util.Map;

public class ConfigurationCache {
    private static ConfigurationCache instancia;
    private Map<String, String> configuracoes;

    private ConfigurationCache() {
        this.configuracoes = new HashMap<>();
        System.out.println("Lendo variáveis do sistema (Disco/S3)...");
        this.configuracoes.put("api.timeout", "5000");
    }

    public static synchronized ConfigurationCache getInstancia() {
        if (instancia == null) {
            instancia = new ConfigurationCache();
        }
        return instancia;
    }

    public String getValor(String chave) {
        return configuracoes.get(chave);
    }
}
```

2. Padrão Factory Method

Delega a instanciação de objetos para subclasses, permitindo que o código cliente utilize uma interface genérica sem se acoplar às classes concretas.

Cenário 2.1: Processamento de Diferentes Formatos de Arquivo

Contexto: Um sistema de integração de dados (ETL) recebe arquivos diários em formatos variados. A regra de negócio para importar dados é a mesma, mas a forma de ler os arquivos difere. O Factory Method isola a criação do analisador (*parser*) específico.

```

// O Produto
public interface ArquivoParser { void processar(String caminho); }

// Produtos Concretos
public class CSVParser implements ArquivoParser {
    public void processar(String caminho) { System.out.println("Processando CSV: " + caminho); }
}
public class JSONParser implements ArquivoParser {
    public void processar(String caminho) { System.out.println("Processando JSON: " + caminho); }
}

// O Criador (A Fábrica)
public abstract class ImportadorDados {
    protected abstract ArquivoParser criarParser(); // Factory Method

    public void importar(String caminho) {
        ArquivoParser parser = criarParser();
        System.out.println("Iniciando auditoria de importação...");
        parser.processar(caminho);
    }
}

// Criadores Concretos
public class ImportadorCSV extends ImportadorDados {
    protected ArquivoParser criarParser() { return new CSVParser(); }
}

```

Cenário 2.2: Canais de Notificação Omnichannel

Contexto: Um módulo de CRM envia alertas aos clientes. O canal (E-mail, SMS, Push Notification) depende da preferência do usuário. O Factory Method garante que novos canais possam ser adicionados futuramente sem alterar a lógica central de campanhas do CRM.

Java

```

public interface CanalNotificacao { void enviar(String destinatario, String mensagem); }

```

```
public class NotificacaoSMS implements CanalNotificacao {
    public void enviar(String dest, String msg) { System.out.println("SMS para " + dest + ": " + msg); }
}
```

```
public abstract class CampanhaCRM {
    protected abstract CanalNotificacao criarCanal(); // Factory Method

    public void dispararCampanha(String destinatario, String mensagem) {
        CanalNotificacao canal = criarCanal();
        canal.enviar(destinatario, mensagem);
    }
}
```

```
public class CampanhaMobile extends CampanhaCRM {
    protected CanalNotificacao criarCanal() { return new NotificacaoSMS(); }
}
```

Cenário 2.3: Geração de Relatórios por Nível de Acesso

Contexto: Um ERP possui relatórios de vendas. O relatório gerado para um gerente possui dados estratégicos, enquanto o gerado para um vendedor possui apenas dados operacionais. A fábrica decide qual objeto Relatorio criar baseada no perfil do solicitante.

Java

```
public interface Relatorio { void exibirDados(); }
```

```
public class RelatorioOperacional implements Relatorio {
    public void exibirDados() { System.out.println("Dados Básicos de Venda."); }
}
```

```
public class RelatorioEstrategico implements Relatorio {
    public void exibirDados() { System.out.println("Dados de Lucro e DRE."); }
}
```

```
public abstract class EmissorRelatorio {
    protected abstract Relatorio fabricarRelatorio(); // Factory Method

    public void emitir() {
```

```

    Relatorio r = fabricarRelatorio();
    r.exibirDados();
}
}

public class EmissorGerencial extends EmissorRelatorio {
    protected Relatorio fabricarRelatorio() { return new RelatorioEstrategico(); }
}

```

3. Padrão Abstract Factory

Fornece uma interface para criar famílias de objetos correlacionados ou dependentes sem especificar suas classes concretas.

Cenário 3.1: Provedores de Nuvem (Cloud Agnostic)

Contexto: O sistema da organização precisa realizar o provisionamento automático de infraestrutura. Dependendo do contrato ativo, a infraestrutura pode ser provisionada na AWS ou no Microsoft Azure. A fábrica abstrata garante que uma máquina virtual da AWS não seja conectada a um banco de dados nativo do Azure.

Java

```

// Interfaces da Família
public interface VirtualMachine { void inicializar(); }
public interface StorageBlob { void salvar(String arquivo); }

// Produtos AWS
public class EC2 implements VirtualMachine { public void inicializar() { System.out.println("AWS EC2 Up"); } }
public class S3 implements StorageBlob { public void salvar(String arq) { System.out.println("Salvo no S3"); } }

// A Fábrica Abstrata
public interface CloudProviderFactory {
    VirtualMachine criarVM();
    StorageBlob criarStorage();
}

```

```
// Fábrica Concreta
public class AWSFactory implements CloudProviderFactory {
    public VirtualMachine criarVM() { return new EC2(); }
    public StorageBlob criarStorage() { return new S3(); }
}
```

Cenário 3.2: Exportação de Documentos (PDF vs HTML)

Contexto: Um sistema gera faturas corporativas. Uma fatura é composta por um "Cabeçalho" e um "Corpo". Se a exportação for para impressão, cria-se a família PDF. Se for para visualização web, cria-se a família HTML.

Java

```
public interface Cabecalho { void renderizarTopo(); }
public interface Corpo { void renderizarConteudo(); }
```

```
public class CabecalhoPDF implements Cabecalho { public void renderizarTopo() { /* Lógica PDF */ } }
public class CorpoPDF implements Corpo { public void renderizarConteudo() { /* Lógica PDF */ } }
```

```
public interface FaturaFactory {
    Cabecalho criarCabecalho();
    Corpo criarCorpo();
}
```

```
public class PDFFactory implements FaturaFactory {
    public Cabecalho criarCabecalho() { return new CabecalhoPDF(); }
    public Corpo criarCorpo() { return new CorpoPDF(); }
}
```

Cenário 3.3: Componentes de Banco de Dados Relacional

Contexto: A aplicação precisa suportar múltiplos bancos de dados (MySQL e Oracle) de forma transparente. A fábrica garante que a aplicação crie conexões e comandos (Commands) que pertençam à mesma família de driver (ex: não usar um comando Oracle em uma conexão MySQL).

Java

```
public interface ConexaoBD { void conectar(); }
public interface ComandoBD { void executar(); }

public class ConexaoOracle implements ConexaoBD { public void conectar() { /* ... */ } }
public class ComandoOracle implements ComandoBD { public void executar() { /* ... */ } }

public interface DatabaseFactory {
    ConexaoBD criarConexao();
    ComandoBD criarComando();
}

public class OracleFactory implements DatabaseFactory {
    public ConexaoBD criarConexao() { return new ConexaoOracle(); }
    public ComandoBD criarComando() { return new ComandoOracle(); }
}
```

4. Padrão Builder

Separa a construção de um objeto complexo de sua representação final, contornando o problema de construtores excessivamente longos e difíceis de ler.

Cenário 4.1: Construtor de Requisições de Rede (HTTP)

Contexto: Fazer uma chamada de API exige configurar URL, método, cabeçalhos de segurança, tempo limite e corpo da requisição. Passar tudo via construtor gera código confuso. O Builder fornece uma interface fluente e legível.

Java

```
public class HttpRequest {
    private String url; private String method; private String body;

    private HttpRequest(Builder builder) {
        this.url = builder.url; this.method = builder.method; this.body = builder.body;
    }
}
```



```

    }

    public static class Builder {
        private String url; private String method = "GET"; private String body;

        public Builder(String url) { this.url = url; }
        public Builder method(String method) { this.method = method; return this; }
        public Builder body(String body) { this.body = body; return this; }

        public HttpRequest build() { return new HttpRequest(this); }
    }
}

// Uso: new HttpRequest.Builder("http://api.com").method("POST").body("{v:1}").build();

```

Cenário 4.2: Construção de Redes Neurais Artificiais

Contexto: A configuração de um modelo de Inteligência Artificial requer a definição do número de camadas ocultas, taxa de aprendizado, função de ativação e algoritmo otimizador. O Builder facilita a montagem etapa por etapa desta topologia complexa antes de iniciar o treinamento.

Java

```

public class NeuralNetworkModel {
    private int hiddenLayers; private double learningRate; private String optimizer;

    private NeuralNetworkModel(Builder b) {
        this.hiddenLayers = b.hiddenLayers; this.learningRate = b.learningRate; this.optimizer =
b.optimizer;
    }

    public static class Builder {
        private int hiddenLayers = 1; private double learningRate = 0.01; private String optimizer =
"SGD";

        public Builder addHiddenLayers(int layers) { this.hiddenLayers = layers; return this; }
        public Builder setLearningRate(double lr) { this.learningRate = lr; return this; }
        public Builder setOptimizer(String opt) { this.optimizer = opt; return this; }
    }
}

```

```

    public NeuralNetworkModel compile() { return new NeuralNetworkModel(this); }
}
}

```

Cenário 4.3: Montagem de Contratos Eletrônicos

Contexto: Em um sistema jurídico, um documento pode possuir dezenas de cláusulas opcionais (fiador, seguro, multas rescisórias). O Builder garante que o contrato seja validado e construído apenas com as seções pertinentes.

Java

```

public class ContratoJuridico {
    private String contratante; private boolean possuiFiador; private double multaRescisoria;

    private ContratoJuridico(Builder b) {
        this.contratante = b.contratante; this.possuiFiador = b.possuiFiador; this.multaRescisoria =
b.multaRescisoria;
    }

    public static class Builder {
        private String contratante; private boolean possuiFiador = false; private double
multaRescisoria = 0.0;

        public Builder comContratante(String nome) { this.contratante = nome; return this; }
        public Builder incluirFiador() { this.possuiFiador = true; return this; }
        public Builder aplicarMulta(double valor) { this.multaRescisoria = valor; return this; }

        public ContratoJuridico assinar() { return new ContratoJuridico(this); }
    }
}

```

5. Padrão Prototype

Cria novos objetos copiando uma instância pré-existente (o protótipo), em vez de instanciar a partir do zero. Otimiza cenários onde a criação consome muita CPU ou acessos ao banco de

dados.

Cenário 5.1: Geração de Entidades em Motores Gráficos (Spawning)

Contexto: Em um jogo, invocar 100 inimigos idênticos simultaneamente, carregando texturas e atributos do disco para cada um, causa travamentos (*drops de frame*). O motor carrega um "Inimigo Protótipo" na inicialização e o clona 100 vezes, alterando apenas a posição (X, Y) na tela.

Java

```
public interface CloneableEntity { CloneableEntity cloneEntity(); }

public class InimigoNPC implements CloneableEntity {
    private String texturaComplexa; // Arquivo pesado
    private int posX, posY;

    public InimigoNPC(String textura) {
        // Simulação de carregamento de arquivo de 50MB
        this.texturaComplexa = textura;
    }

    private InimigoNPC(InimigoNPC source) {
        this.texturaComplexa = source.texturaComplexa; // Reaproveita referência
    }

    public InimigoNPC cloneEntity() { return new InimigoNPC(this); }
    public void setPosition(int x, int y) { this.posX = x; this.posY = y; }
}
```

Cenário 5.2: Duplicação de Roteiros de Viagem

Contexto: Em um sistema de agência de turismo, um "Pacote Europa 15 dias" possui um itinerário massivo (voos, hotéis, traslados). Quando um cliente deseja comprar esse pacote, mas quer alterar apenas o hotel do último dia, o sistema clona o pacote original e aplica a mutação apenas no clone.

Java

```

public class PacoteViagem implements CloneableEntity {
    private String itinerarioBase;
    private String alteracoesCustomizadas;

    public PacoteViagem(String itinerarioBase) {
        // Simula processamento de múltiplas APIs aéreas
        this.itinerarioBase = itinerarioBase;
    }
    private PacoteViagem(PacoteViagem source) {
        this.itinerarioBase = source.itinerarioBase;
    }
    public PacoteViagem cloneEntity() { return new PacoteViagem(this); }
    public void setCustomizacao(String alteracao) { this.alteracoesCustomizadas = alteracao; }
}

```

Cenário 5.3: Propagação de Configurações de Servidor

Contexto: Ao provisionar máquinas virtuais em um cluster (ex: Kubernetes), o estado inicial (variáveis de ambiente, regras de firewall) é idêntico. Em vez de calcular as regras para cada novo *pod/node*, instancia-se um nó protótipo e clona-se a configuração para os demais, alterando apenas o endereço IP.

Java

```

public class NodeConfigurator implements CloneableEntity {
    private String regrasFirewall;
    private String ipAddress;

    public NodeConfigurator() {
        // Processamento heurístico de segurança
        this.regrasFirewall = "BLOCK_ALL_EXCEPT_80_443";
    }
    private NodeConfigurator(NodeConfigurator source) {
        this.regrasFirewall = source.regrasFirewall;
    }
    public NodeConfigurator cloneEntity() { return new NodeConfigurator(this); }
    public void setIpAddress(String ip) { this.ipAddress = ip; }
}

```

}